

Implementación de un Proceso DevOps para un Proyecto Java con Despliegue en AWS, Amazon RDS, Docker Compose y GitHub Actions

Emilio Quinde
Cursando Sistemas Computacionales
Universidad Católica de Cuenca
Cuenca-Ecuador
emilio.quinde@est.ucacue.edu.ec

Adrián Bravo
Cursando Sistemas Computacionales
Universidad Católica de Cuenca
Cuenca-Ecuador
adrian.bravo@est.ucacue.edu.ec

Kevin Marca
Cursando Sistemas Computacionales
Universidad Católica de Cuenca
Cuenca-Ecuador
kevin.marca@est.ucacue.edu.ec

Resumen—El presente informe describe la implementación de un proceso DevOps completo aplicado a un proyecto Java existente. El trabajo incluyó la configuración de infraestructura en Amazon Web Services mediante una VPC personalizada, una instancia EC2 pública, una base de datos Amazon RDS PostgreSQL privada, reglas de seguridad, pruebas de conectividad, ejecución local mediante Docker Compose, publicación del proyecto en GitHub y automatización CI/CD con GitHub Actions. El proceso permitió validar la comunicación entre EC2 y RDS, impedir el acceso directo a la base de datos desde DBeaver, construir y probar el proyecto Java con Gradle, desplegar la aplicación en EC2 y comprobar el funcionamiento del endpoint público mediante Postman. Todo el flujo fue documentado mediante evidencias visuales de cada etapa, desde la creación de la red en AWS hasta las pruebas finales del endpoint REST.

Abstract— This report describes the implementation of a complete DevOps process applied to an existing Java project. The work included configuring the infrastructure on Amazon Web Services using a custom VPC, a public EC2 instance, a private Amazon RDS PostgreSQL database, security rules, connectivity testing, local execution using Docker Compose, publishing the project to GitHub, and CI/CD automation with GitHub Actions. The process allowed us to validate communication between EC2 and RDS, prevent direct access to the database from DBeaver, build and test the Java project with Gradle, deploy the application to EC2, and verify the functionality of the public endpoint using Postman. The entire workflow was documented with visual evidence of each stage, from network creation on AWS to final testing of the REST endpoint.

Palabras clave—DevOps, AWS, EC2, RDS, VPC, Docker Compose, GitHub Actions, Java, Spring Boot, PostgreSQL, CI/CD.

keywords— DevOps, AWS, EC2, RDS, VPC, Docker Compose, GitHub Actions, Java, Spring Boot, PostgreSQL, CI/CD.

I. INTRODUCCIÓN

La aplicación de prácticas DevOps permite mejorar la forma en que se desarrollan, prueban y despliegan sistemas de software. En un entorno tradicional, el desarrollo y la operación de una aplicación suelen manejarse como procesos separados, lo cual puede generar retrasos, errores manuales y poca trazabilidad. DevOps propone integrar estas etapas mediante

automatización, control de versiones, pruebas continuas y despliegues repetibles.

En este proyecto se implementó un proceso DevOps para un sistema Java existente, el cual fue descargado desde un repositorio base y posteriormente modificado para conectarse a una base de datos PostgreSQL. La aplicación fue probada de forma local, contenerizada mediante Docker Compose, subida a un repositorio GitHub y desplegada automáticamente en una instancia EC2 mediante GitHub Actions.

La infraestructura se diseñó en AWS siguiendo una separación entre recursos públicos y privados. La instancia EC2 fue ubicada en una subred pública para exponer el servicio Java por medio del puerto 8080. Por otro lado, la base de datos Amazon RDS fue ubicada en subredes privadas, sin acceso público, permitiendo conexión únicamente desde la instancia EC2. Esta decisión permitió cumplir con uno de los requisitos principales del proyecto: que la base de datos no pueda ser accedida directamente desde DBeaver.

El informe presenta paso a paso la implementación realizada, integrando todas las evidencias visuales obtenidas durante la ejecución del proyecto. De esta manera, cada captura se relaciona con la explicación técnica del proceso y no queda separada como un anexo aislado.

II. OBJETIVOS

A. Objetivo general

Implementar un proceso DevOps completo para un proyecto Java existente, integrando infraestructura en AWS, base de datos Amazon RDS, pruebas locales con Docker Compose, repositorio GitHub y automatización CI/CD mediante GitHub Actions.

B. Objetivos específicos

- Crear una VPC personalizada para alojar los recursos del proyecto.
- Configurar una instancia EC2 con el nombre GRUPO_5_EC2.

- Crear una base de datos Amazon RDS con el nombre GRUPO-5-RDS.
- Diseñar una arquitectura con una subred pública para EC2 y subredes privadas para RDS.
- Configurar reglas de seguridad que permitan comunicación EC2-RDS sin exponer la base de datos.
- Validar la conexión entre EC2 y Amazon RDS mediante psql.
- Comprobar que no sea posible acceder a RDS desde DBEaver.
- Modificar el proyecto Java para conectarse a PostgreSQL.
- Ejecutar pruebas locales con Gradle y Docker Compose.
- Publicar el proyecto en GitHub con el formato GRUPO_5_DEVOPS.
- Crear un pipeline CI/CD con los jobs build, test y deploy.
- Desplegar automáticamente el proyecto en EC2.
- Validar el endpoint público de la aplicación mediante Postman.

III. MARCO TEÓRICO

A. DevOps

DevOps es un enfoque de trabajo que busca integrar el desarrollo de software con las operaciones de infraestructura. Su propósito es reducir la separación entre el equipo que construye una aplicación y el equipo que la despliega o mantiene en ejecución. En este proyecto, DevOps se aplicó mediante el uso conjunto de Git, GitHub, Docker, Docker Compose, AWS y GitHub Actions. Cada herramienta cumplió una función dentro del ciclo de vida del software.

B. Computación en la nube

La computación en la nube permite utilizar recursos tecnológicos a través de Internet, sin necesidad de administrar físicamente servidores. AWS es la plataforma de nube utilizada en este proyecto, mediante sus servicios Amazon VPC, EC2 y RDS, los cuales permitieron construir una arquitectura donde la aplicación Java se ejecuta en la nube y se comunica con una base de datos administrada.

C. Amazon VPC

Amazon Virtual Private Cloud permite lanzar recursos de AWS dentro de una red virtual aislada. La VPC GRUPO_5_VPC fue configurada con el bloque CIDR 10.0.0.0/16, lo cual permitió dividir la red en subredes públicas y privadas y controlar de manera explícita qué recursos tendrían acceso a Internet.

D. Subredes públicas y privadas

Una subred es un rango de direcciones IP dentro de una VPC. La subred pública (10.0.1.0/24) alojó a EC2 con acceso a Internet por medio del Internet Gateway. Las subredes privadas (10.0.2.0/24 y 10.0.3.0/24) alojaron a Amazon RDS sin acceso directo desde Internet.

E. Internet Gateway y tablas de rutas

El Internet Gateway permite conectar una VPC con Internet. Las tablas de rutas determinan hacia dónde se dirige el tráfico de red. La tabla pública GRUPO_5_PUBLIC_RT contiene la ruta 0.0.0.0/0 hacia el Internet Gateway, mientras que la tabla privada GRUPO_5_PRIVATE_RT mantiene únicamente la ruta local.

F. Security Groups

Los Security Groups funcionan como firewalls virtuales. El Security Group de EC2 (GRUPO_5_EC2_SG) permitió tráfico SSH (puerto 22) y HTTP (puerto 8080). El Security Group de RDS (GRUPO_5_RDS_SG) permitió PostgreSQL (puerto 5432) únicamente desde el SG de EC2, evitando así la exposición pública de la base de datos.

G. Amazon EC2 y llaves SSH

Amazon EC2 permite crear servidores virtuales. La instancia GRUPO_5_EC2 fue ubicada en la subred pública y accedida mediante SSH usando una llave privada PEM (grupo_5_key). En EC2 se instalaron Docker, Git y Docker Compose para soportar el despliegue.

H. Amazon RDS y PostgreSQL

Amazon RDS es un servicio administrado para bases de datos relacionales. Se utilizó para crear una instancia PostgreSQL identificada como GRUPO-5-RDS, sin acceso público y dentro de subredes privadas. PostgreSQL fue utilizado tanto en la nube (RDS) como en local (Docker Compose) para almacenar la información de los usuarios registrados.

I. Spring Boot y Gradle

Spring Boot facilita la creación de aplicaciones Java con servidores embebidos como Tomcat, escuchando en el puerto 8080. Gradle se utilizó para construir y probar el proyecto: gradlew build y gradlew test. Se declaró la dependencia del driver PostgreSQL 42.7.3 en build.gradle.

J. Docker y Docker Compose

Docker permite empaquetar la aplicación y sus dependencias en contenedores portables. Docker Compose orquesta múltiples contenedores mediante un archivo YAML; en este proyecto se levantaron dos servicios (PostgreSQL y la aplicación Java) con un solo comando.

K. Git, GitHub y GitHub Actions

Git permitió controlar versiones del código. GitHub alojó el repositorio remoto GRUPO_5_DEVOPS. GitHub Actions automatizó el pipeline CI/CD mediante el archivo .github/workflows/ci.yml, ejecutando los jobs build, test y deploy ante cada push a la rama main.

L. CI/CD y Secrets

CI/CD significa Integración Continua y Despliegue Continuo. Los secrets de GitHub (EC2_HOST, EC2_USER, EC2_KEY) almacenaron información sensible como la IP de

EC2, el usuario SSH y la llave privada, evitando exponer credenciales en el código.

IV. DESARROLLO E IMPLEMENTACIÓN

A. Diseño y creación de la VPC

El primer paso fue acceder al panel de Amazon VPC. La consola mostró el listado inicial de VPCs disponibles en la cuenta, donde se identificó la VPC por defecto antes de comenzar la creación personalizada.

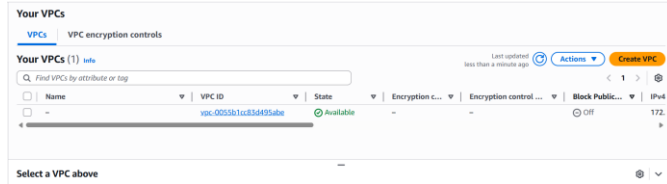


Figura 1. Listado inicial de VPCs disponibles en la cuenta antes de crear la VPC del proyecto.

A continuación se inició el formulario "Create VPC" seleccionando la opción "VPC only", asignando el nombre GRUPO_5_VPC y configurando el bloque CIDR IPv4 manual con valor 10.0.0.0/16.

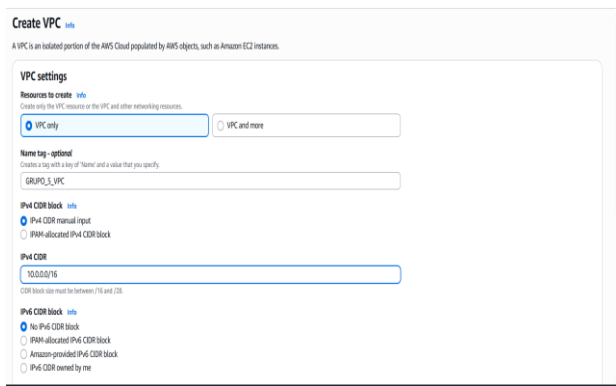


Figura 2. Formulario de creación de la VPC GRUPO_5_VPC con bloque CIDR 10.0.0.0/16.

AWS confirmó la creación de la VPC, asignándole un identificador único (vpc-06dedcbf01eeceeb5) y dejándola en estado Available.

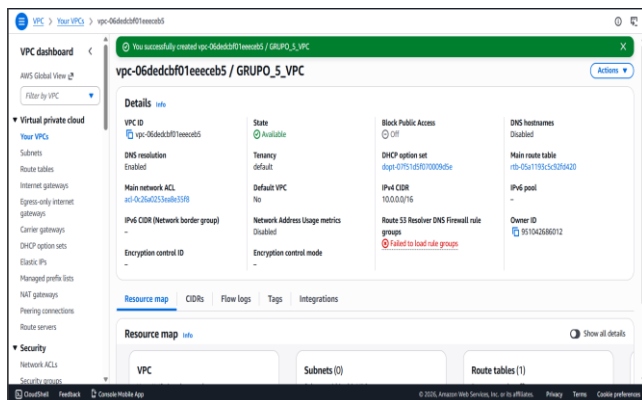


Figura 3. VPC GRUPO_5_VPC creada exitosamente en estado Available.

Luego de crear la VPC, se procedió a crear las subredes. La consola "Create subnet" permitió seleccionar la VPC recién creada como base para las subredes derivadas.

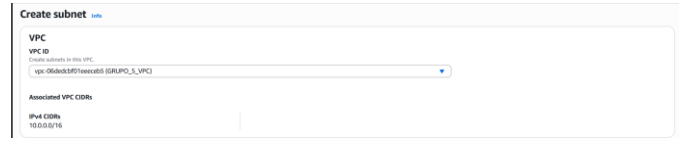


Figura 4. Selección de la VPC GRUPO_5_VPC al iniciar la creación de subredes.

La primera subred creada fue GRUPO_5_PUBLIC_SUBNET en la zona us-east-1a con el bloque CIDR 10.0.1.0/24. Esta subred fue destinada a la instancia EC2.

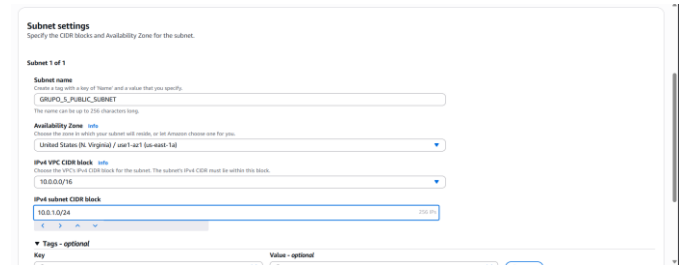


Figura 5. Configuración de la subred pública GRUPO_5_PUBLIC_SUBNET con CIDR 10.0.1.0/24.

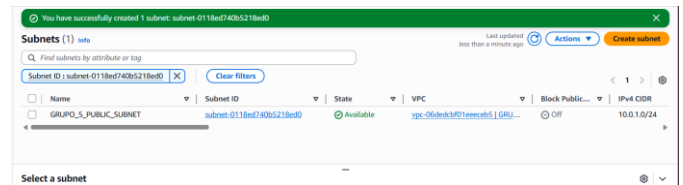


Figura 6. Subred pública GRUPO_5_PUBLIC_SUBNET creada exitosamente en estado Available.

Posteriormente se crearon dos subredes privadas. La primera fue GRUPO_5_PRIVATE_SUBNET_A en us-east-1a con CIDR 10.0.2.0/24.

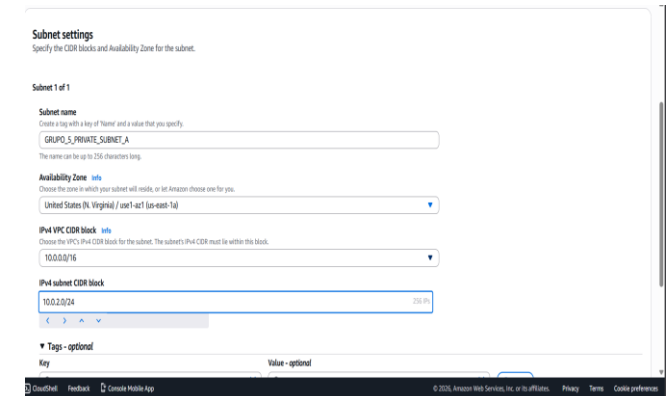


Figura 7. Configuración de la subred privada GRUPO_5_PRIVATE_SUBNET_A con CIDR 10.0.2.0/24.

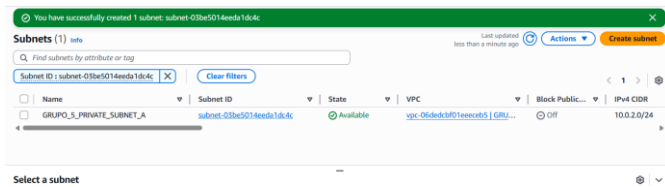


Figura 8. Subred privada A creada exitosamente.

La segunda subred privada GRUPO_5_PRIVATE_SUBNET_B se creó en una zona de disponibilidad diferente (us-east-1b) con CIDR 10.0.3.0/24, requisito necesario para que Amazon RDS pueda operar en alta disponibilidad.

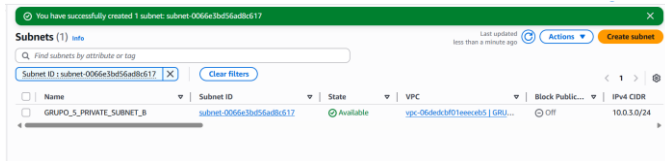


Figura 9. Subred privada B creada exitosamente en una zona de disponibilidad distinta.

El listado final mostró las tres subredes asociadas a la VPC del proyecto: una pública y dos privadas.

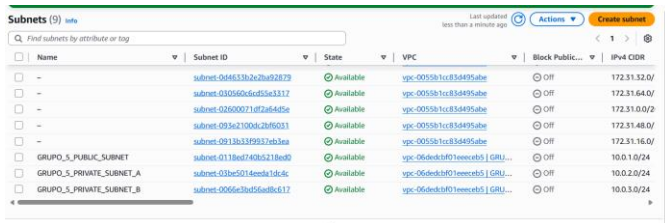


Figura 10. Listado final de las subredes creadas: GRUPO_5_PUBLIC_SUBNET, PRIVATE_SUBNET_A y PRIVATE_SUBNET_B.

B. Internet Gateway y tabla de rutas pública

Para que la subred pública pudiera comunicarse con Internet, se accedió a la sección "Internet gateways" del panel VPC. Inicialmente se observó el listado existente para identificar gateways previos.

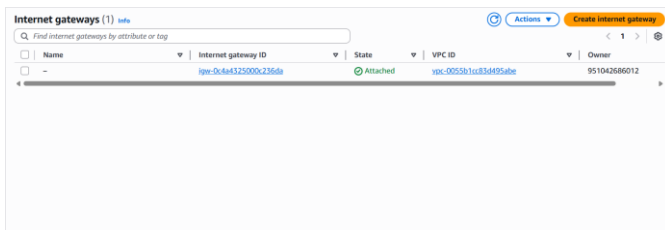


Figura 11. Listado de Internet Gateways existentes antes de crear el del proyecto.

Se creó un nuevo Internet Gateway llamado GRUPO_5_IGW, agregando además tags descriptivas del proyecto.

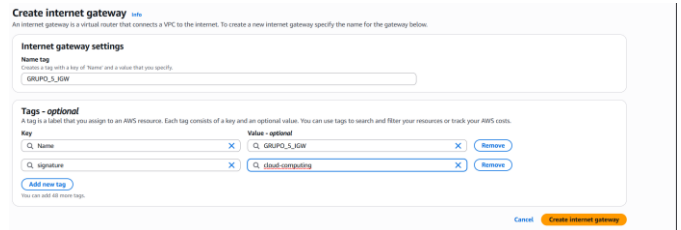


Figura 12. Formulario de creación del Internet Gateway GRUPO_5_IGW.

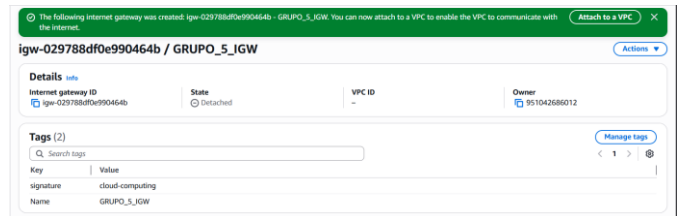


Figura 13. Internet Gateway GRUPO_5_IGW creado en estado Detached, pendiente de asociar a la VPC.

A continuación se asoció el Internet Gateway a la VPC GRUPO_5_VPC mediante la opción "Attach to a VPC".

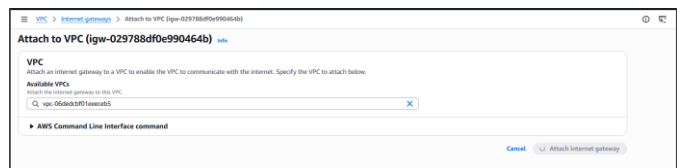


Figura 14. Asociación del Internet Gateway GRUPO_5_IGW a la VPC GRUPO_5_VPC.

Una vez disponible el Internet Gateway, se procedió a crear la tabla de rutas pública. Primero se revisó el listado de tablas de rutas existentes.

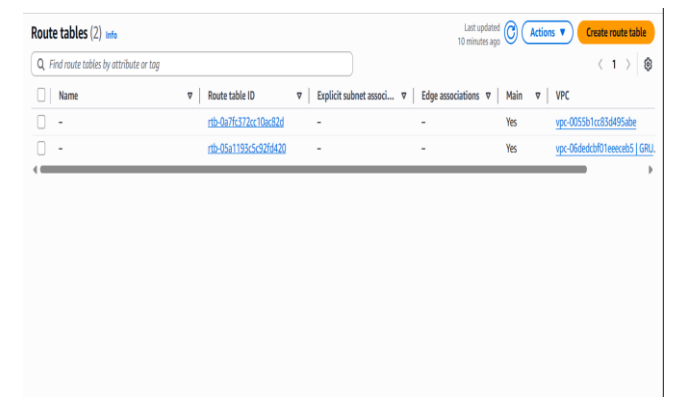


Figura 15. Listado inicial de tablas de rutas en la VPC.

Se creó la tabla de rutas GRUPO_5_PUBLIC_RT asociada a la VPC del proyecto.

Create route table

A route table specifies how packets are forwarded between the subnets within your VPC, the Internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

GRUPO_5_PUBLIC_RT

VPC
This VPC is used for this route table.

vpc-06d6c3f01eeec6b5 (GRUPO_5_VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key Value - optional

Q Name X Q GRUPO_5_PUBLIC_RT X Remove

Q signature X Q cloud-computing X Remove

Add new tag

You can add up to 50 tags.

Cancel Create route table

Figura 16. Formulario de creación de la tabla de rutas pública GRUPO_5_PUBLIC_RT.

Route table rtb-0a2c090db78dd68d5 / GRUPO_5_PUBLIC_RT was created successfully.

rtb-0a2c090db78dd68d5 / GRUPO_5_PUBLIC_RT

Details

Route table ID
rtb-0a2c090db78dd68d5

VPC
vpc-06d6c3f01eeec6b5 (GRUPO_5_VPC)

Owner ID
951042686012

Routes (1)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Figura 17. Tabla de rutas GRUPO_5_PUBLIC_RT creada con la ruta local 10.0.0.0/16 activa.

Para permitir salida hacia Internet, se editaron las rutas agregando 0.0.0.0/0 con destino al Internet Gateway GRUPO_5_IGW.

Edit routes

Destination: 10.0.0.0/16

Target: local

Status: Active

Propagated: No

Route Origin: CreateRouteTable

Add route

Routes (1)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable

Figura 18. Edición de rutas: se agrega 0.0.0.0/0 con destino al Internet Gateway.

Updated routes for rtb-0a2c090db78dd68d5 / GRUPO_5_PUBLIC_RT successfully

rtb-0a2c090db78dd68d5 / GRUPO_5_PUBLIC_RT

Details

Route table ID
rtb-0a2c090db78dd68d5

VPC
vpc-06d6c3f01eeec6b5 (GRUPO_5_VPC)

Owner ID
951042686012

Routes (2)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-029718a0f990464b	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Figura 19. Tabla de rutas pública actualizada con dos rutas activas: local y 0.0.0.0/0 hacia el IGW.

C. Tabla de rutas privada y asociación de subredes

Para las subredes privadas se creó una tabla de rutas independiente llamada GRUPO_5_PRIVATE_RT, sin ruta hacia Internet.

Create route table

A route table specifies how packets are forwarded between the subnets within your VPC, the Internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

GRUPO_5_PRIVATE_RT

VPC
This VPC is used for this route table.

vpc-06d6c3f01eeec6b5 (GRUPO_5_VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key Value - optional

Q Name X Q GRUPO_5_PRIVATE_RT X Remove

Q signature X Q cloud-computing X Remove

Add new tag

You can add up to 50 tags.

Cancel Create route table

Figura 20. Formulario de creación de la tabla de rutas privada GRUPO_5_PRIVATE_RT.

Route table rtb-0271a59f5e1cf87a / GRUPO_5_PRIVATE_RT was created successfully.

rtb-0271a59f5e1cf87a / GRUPO_5_PRIVATE_RT

Details

Route table ID
rtb-0271a59f5e1cf87a

VPC
vpc-06d6c3f01eeec6b5 (GRUPO_5_VPC)

Owner ID
951042686012

Routes (1)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Figura 21. Tabla de rutas privada creada con una sola ruta local 10.0.0.0/16.

Inicialmente la tabla no tenía subredes asociadas, por lo que se utilizó la opción "Edit subnet associations".

Route table rtb-0271a59f5e1cf87a / GRUPO_5_PRIVATE_RT was created successfully.

rtb-0271a59f5e1cf87a / GRUPO_5_PRIVATE_RT

Subnet associations

Explicit subnet associations (0)

No subnet associations
You do not have any subnet associations.

Subnets without explicit associations (2)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
GRUPO_5_PRIVATE_SUBNET_A	subnet-03be0511a6da1846	10.0.2.0/24	-
GRUPO_5_PRIVATE_SUBNET_B	subnet-0066c34fd6db6117	10.0.3.0/24	-

Figura 22. Tabla privada sin asociaciones explícitas antes de vincular las subredes.

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (2/3)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
GRUPO_5_PUBLIC_SUBNET	subnet-0118e7436c2118e0	10.0.1.0/24	-	rtb-0a2c090db78dd68d5 / GRUPO_5_PUBLIC_RT
GRUPO_5_PRIVATE_SUBNET_A	subnet-03be0511a6da1846	10.0.2.0/24	-	Main (rtb-01a11915c1029420)
GRUPO_5_PRIVATE_SUBNET_B	subnet-0066c34fd6db6117	10.0.3.0/24	-	Main (rtb-01a11915c1029420)

Selected subnets

subnet-03be0511a6da1846 / GRUPO_5_PRIVATE_SUBNET_A X subnet-0066c34fd6db6117 / GRUPO_5_PRIVATE_SUBNET_B X

Cancel Save association

Figura 23. Selección de las subredes privadas A y B para asociarlas a GRUPO_5_PRIVATE_RT.

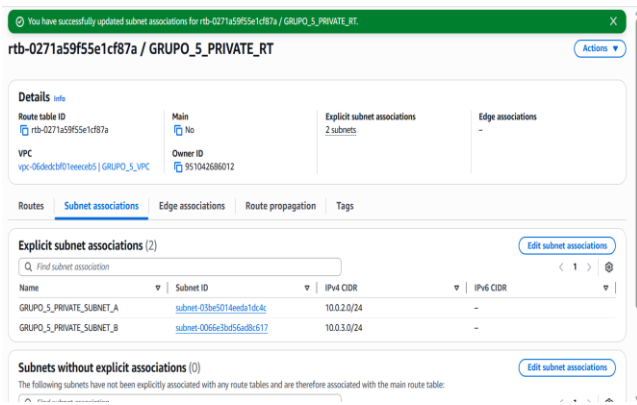


Figura 24. Asociación completada: ambas subredes privadas vinculadas a la tabla privada.

D. Configuración de Security Groups

Después de la red, se configuraron los grupos de seguridad. Primero se revisó el listado de Security Groups existentes para no duplicar reglas.

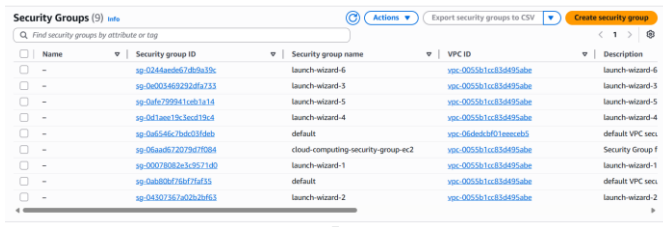


Figura 25. Listado inicial de Security Groups en la cuenta.

Se creó el Security Group GRUPO_5_EC2_SG, asociado a la VPC del proyecto, con la descripción "Security group para EC2 del proyecto DevOps".

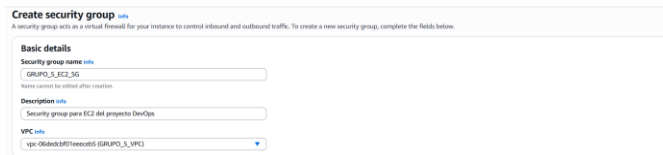


Figura 26. Datos básicos del Security Group GRUPO_5_EC2_SG.

Se configuraron dos reglas de entrada: SSH (puerto 22) desde la IP del administrador y Custom TCP (puerto 8080) para el servicio Java.

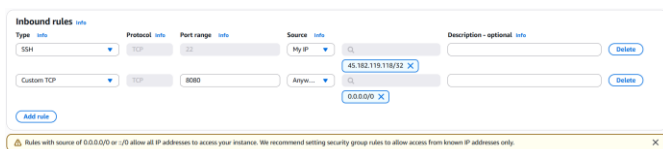


Figura 27. Reglas de entrada del Security Group de EC2: SSH (22) y Custom TCP (8080).

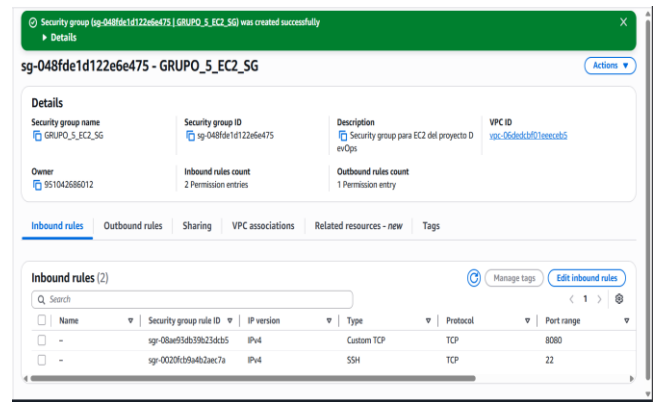


Figura 28. Security Group GRUPO_5_EC2_SG creado con 2 reglas de entrada.

A continuación se creó el Security Group GRUPO_5_RDS_SG, destinado a la base de datos PostgreSQL privada.

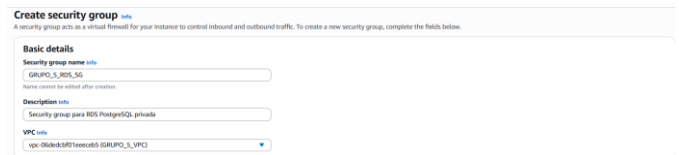


Figura 29. Datos básicos del Security Group GRUPO_5_RDS_SG.

La regla de entrada permitió tráfico PostgreSQL (puerto 5432) únicamente desde el Security Group de EC2, impidiendo conexiones externas directas.



Figura 30. Regla PostgreSQL (5432) restringida al Security Group de EC2.

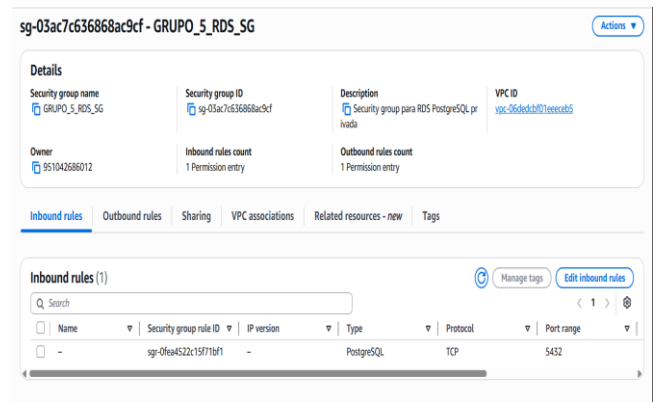


Figura 31. Security Group GRUPO_5_RDS_SG creado correctamente.

E. Creación y configuración de la instancia EC2

Desde el panel de Amazon EC2 se inició el proceso de lanzamiento de una nueva instancia.

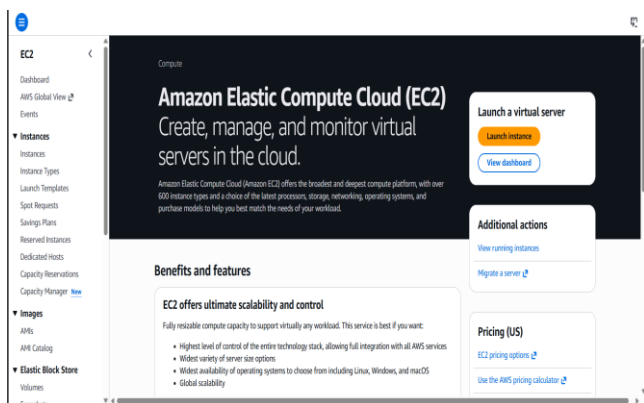


Figura 32. Panel principal de Amazon EC2.

Se asignó el nombre GRUPO_5_EC2 a la nueva instancia, cumpliendo con la nomenclatura del proyecto.

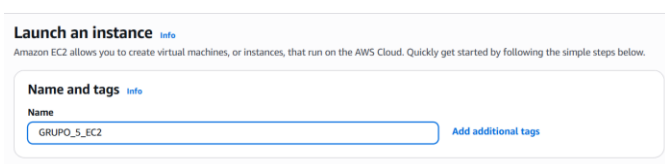


Figura 33. Nombre asignado a la instancia: GRUPO_5_EC2.

La AMI seleccionada fue Ubuntu Server (HVM), SSD Volume Type, dentro de la capa gratuita.

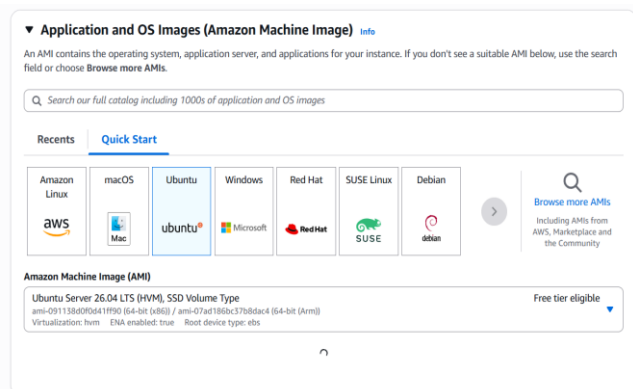


Figura 34. Selección de la AMI Ubuntu Server para la instancia EC2.

El tipo de instancia elegido fue t2.small (1 vCPU, 2 GiB RAM), suficiente para ejecutar la aplicación Java y Docker.

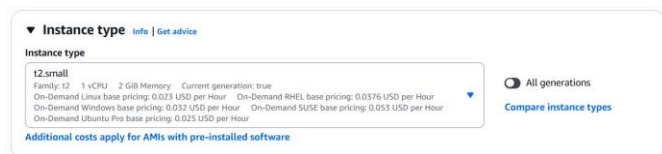


Figura 35. Tipo de instancia EC2 t2.small seleccionado.

Se creó un par de claves SSH llamado grupo_5_key, tipo ED25519 y formato .pem para uso con OpenSSH.

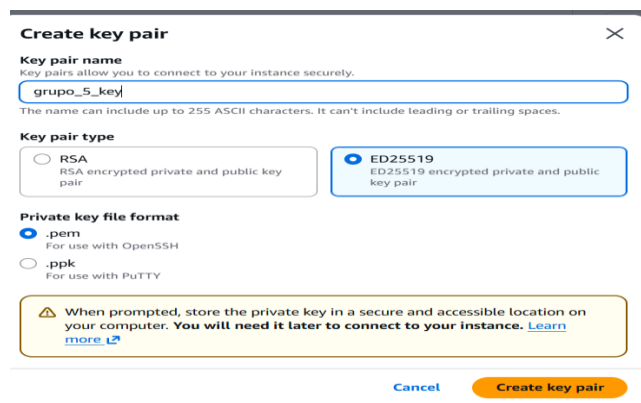


Figura 36. Creación del par de claves SSH grupo_5_key.

La configuración de red asoció la instancia a la VPC GRUPO_5_VPC, la subred pública GRUPO_5_PUBLIC_SUBNET y el Security Group GRUPO_5_EC2_SG, habilitando la asignación automática de IP pública.

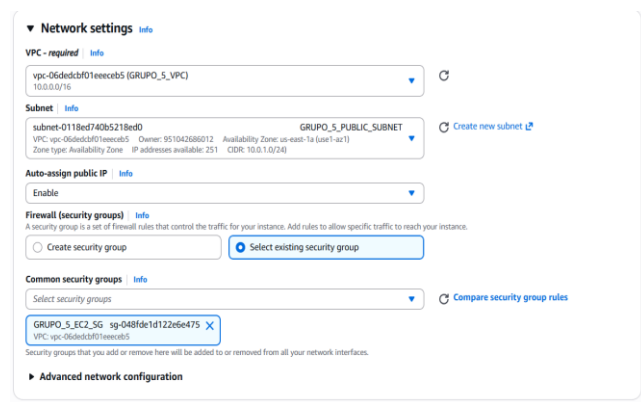


Figura 37. Configuración de red de la instancia: VPC, subred pública, IP pública y Security Group.

Después de lanzar la instancia, ésta apareció en estado Running con su DNS público y la IP 3.238.70.233.

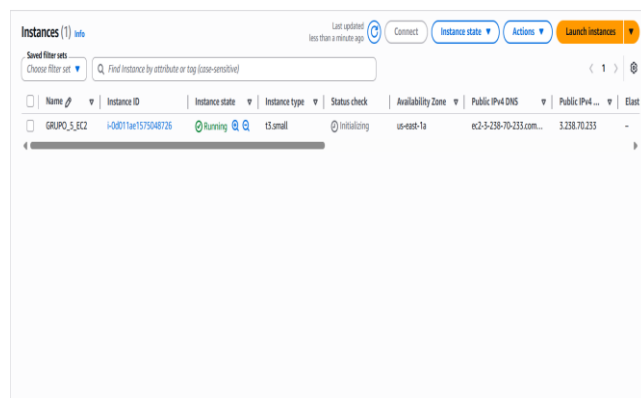


Figura 38. Instancia GRUPO_5_EC2 en estado Running con IPv4 pública asignada.

La conexión inicial se realizó desde Windows mediante SSH usando la llave .pem y la dirección pública del servidor.

Tras aceptar la huella digital, se accedió al sistema Amazon Linux 2023.

```

C:\Users\Usuario\Downloads>icacls .\grupo_5_key.pem /grant:r "MUSERNAME:R"
archivo procesado: .\grupo_5_key.pem
Se procesaron correctamente 1 archivos; error al procesar 0 archivos

C:\Users\Usuario\Downloads>ssh -i .\grupo_5_key.pem ec2-user@3.238.70.233
The authenticity of host '3.238.70.233 (3.238.70.233)' can't be established.
ED25519 key fingerprint is SHA256:BLXXQLI/9bgbQdMgtdiWbex9IcoizdNS/sXQykgMw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '3.238.70.233' (ED25519) to the list of known hosts.

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-0-1-145 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-145 ~]$

```

Figura 39. Conexión SSH exitosa desde Windows hacia la instancia GRUPO_5_EC2.

Dentro de la instancia se instalaron Docker y Git mediante el gestor de paquetes dnf.

```

[ec2-user@ip-10-0-1-145 ~]$ sudo dnf install -y docker git
Last metadata expiration check: 0:00:20 ago on Thu May 14 21:29:28 2026.
Dependencies resolved.
Package                               Architecture Version Repository Size
Installing:
docker                               x86_64     25.0.3-1.0.amzn2023.0.5 amazonlinux 46 M
git                                   x86_64     2.50.1-1.amzn2023.0.1 amazonlinux 53 k
Installing dependencies:
container-selinux                    x86_64     4-2.205-0.1.amzn2023 amazonlinux 58 k
containerd                           x86_64     2.2.3-1.amzn2023.0.1 amazonlinux 26 M
git-core                             x86_64     2.50.1-1.amzn2023.0.1 amazonlinux 2.8 M
libatomic-1.0                        x86_64     1.8.0-3.amzn2023.0.2 amazonlinux 103 k
libtool-ltdl                         x86_64     1.8.8-3.amzn2023.0.2 amazonlinux 70 k
libzstd-devel                        x86_64     1.8.0-2.amzn2023.0.2 amazonlinux 58 k
libzstd-static                       x86_64     1.8.0-2.amzn2023.0.2 amazonlinux 36 k
libzstd                             x86_64     1.8.0-2.amzn2023.0.2 amazonlinux 94 k
perl-error                           x86_64     1.0.17010-2.amzn2023.0.1 amazonlinux 42 k
perl-file-find                       x86_64     1.17-477.amzn2023.0.2 amazonlinux 25 k
perl-git                             x86_64     2.50-1.amzn2023.0.1 amazonlinux 41 k
perl-TermReadKey                     x86_64     3.35-2.amzn2023.0.2 amazonlinux 36 k
perl-lib                             x86_64     0.45-477.amzn2023.0.7 amazonlinux 15 k
perl                                 x86_64     2.5.2-1.amzn2023.0.3 amazonlinux 83 k
perl-core                            x86_64     1.3.0-4.amzn2023.0.2 amazonlinux 3.9 M
Transaction Summary
Install 19 Packages
Total download size: 83 M
Installed size: 326 M
Downloading Packages:
(1/19): container-selinux-4.2.205-0.1.amzn2023.noarch.rpm 1.5 MB/s | 58 kB 00:00
(2/19): git-2.50.1-1.amzn2023.0.1.x86_64.rpm 1.6 MB/s | 53 kB 00:00
(3/19): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm 62 MB/s | 4.9 MB 00:00
(4/19): git-core-doc-2.50.1-1.amzn2023.0.1.noarch.rpm 52 MB/s | 2.8 MB 00:00
(5/19): libatomic-1.8.0-3.amzn2023.0.2.x86_64.rpm 6.2 MB/s | 103 kB 00:00

```

Figura 40. Instalación de Docker y Git en la instancia EC2 mediante dnf.

Se inició el servicio Docker y se verificó su estado activo (running).

```

[ec2-user@ip-10-0-1-145 ~]$ sudo systemctl start docker
[ec2-user@ip-10-0-1-145 ~]$ sudo systemctl status docker
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-05-14 21:33:25 UTC; 7s ago
     TriggeredBy: docker.socket
   Main PID: 28086
   CGroup: /system.slice/docker.service
           └─/usr/lib/docker/daemon --data-root /var/lib/docker --debug --default-ulimit nofile=32768:65536

May 14 21:33:24 ip-10-0-1-145.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine.
May 14 21:33:24 ip-10-0-1-145.ec2.internal docker[28086]: time="2026-05-14T21:33:24.091880000Z" level=info msg="Starting up"
May 14 21:33:25 ip-10-0-1-145.ec2.internal docker[28086]: time="2026-05-14T21:33:25.119773200Z" level=info msg="Loading containers: start."
May 14 21:33:25 ip-10-0-1-145.ec2.internal docker[28086]: time="2026-05-14T21:33:25.140606000Z" level=info msg="Loading containers: done."
May 14 21:33:25 ip-10-0-1-145.ec2.internal docker[28086]: time="2026-05-14T21:33:25.140606000Z" level=info msg="Docker daemon" commit=337795 containerd=
May 14 21:33:25 ip-10-0-1-145.ec2.internal docker[28086]: time="2026-05-14T21:33:25.179718200Z" level=info msg="API listen on /run/docker.sock"
May 14 21:33:25 ip-10-0-1-145.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.

```

Figura 41. Servicio Docker iniciado y funcionando correctamente en EC2.

Se confirmaron las versiones de Docker y Git, y se observó que docker compose aún no estaba disponible como subcomando.

```

[ec2-user@ip-10-0-1-145 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-10-0-1-145 ~]$ git --version
git version 2.50.1
[ec2-user@ip-10-0-1-145 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
[ec2-user@ip-10-0-1-145 ~]$ docker compose version
docker: 'compose' is not a docker command.
See 'docker --help'
[ec2-user@ip-10-0-1-145 ~]$

```

Figura 42. Versiones de Docker y Git instaladas; Docker Compose aún no disponible.

Por ello, se instaló manualmente el plugin de Docker Compose, descargándolo desde el repositorio oficial y dándole permisos de ejecución. Se confirmó la versión Docker Compose v5.1.3.

```

[ec2-user@ip-10-0-1-145 ~]$ sudo mkdir -p /usr/local/lib/docker/cli-plugins
[ec2-user@ip-10-0-1-145 ~]$ sudo curl -L https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose
[ec2-user@ip-10-0-1-145 ~]$ ls -l /usr/local/lib/docker/cli-plugins/docker-compose
-rwxr-xr-x 1 root root 28.5M May 14 21:33 /usr/local/lib/docker/cli-plugins/docker-compose
[ec2-user@ip-10-0-1-145 ~]$ docker compose version
Docker Compose version v5.1.3
[ec2-user@ip-10-0-1-145 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
[ec2-user@ip-10-0-1-145 ~]$

```

Figura 43. Instalación manual del plugin Docker Compose v5.1.3 en EC2.

F. Creación y configuración de Amazon RDS

En el panel de Aurora and RDS, se inició la creación de la base de datos a partir de un estado inicial sin recursos.

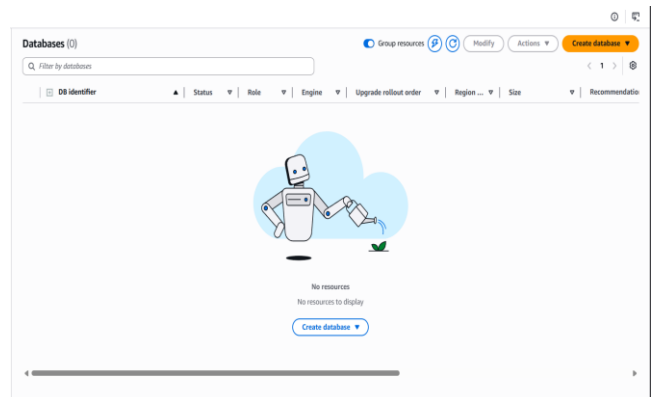


Figura 44. Panel inicial de Amazon Aurora and RDS sin bases de datos creadas.

Se seleccionó el motor PostgreSQL en modalidad estándar (Full configuration).

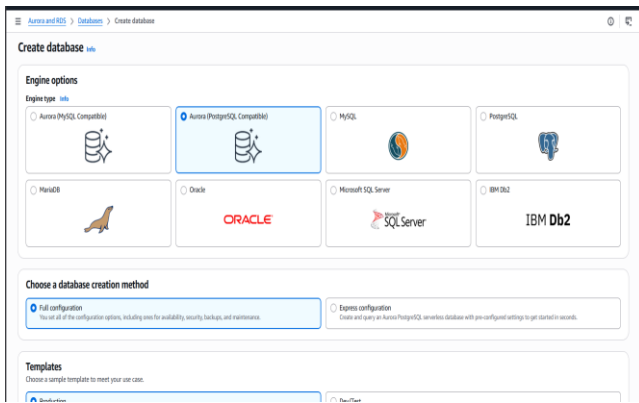


Figura 45. Selección del motor PostgreSQL para la creación de la base de datos.

Se definió el identificador GRUPO-5-RDS, el usuario maestro postgres y la contraseña administrada manualmente.

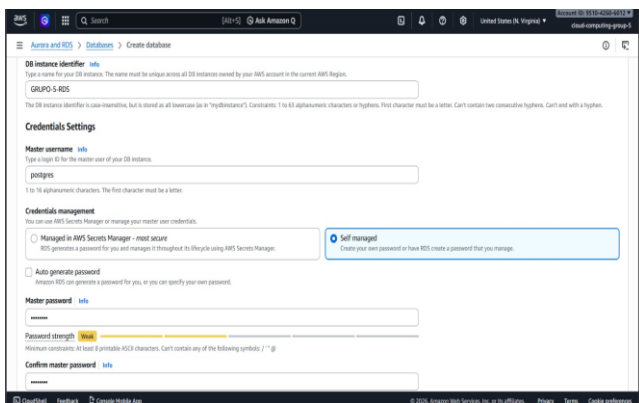


Figura 46. Configuración del identificador GRUPO-5-RDS y credenciales del usuario maestro.

El tipo de instancia elegido fue db.t3.micro, dentro de las clases Burstable (clases t).



Figura 47. Selección del tipo de instancia db.t3.micro para RDS.

En la sección de conectividad se asoció la base a la VPC GRUPO_5_VPC, deshabilitando la opción de acceso público (Public access = No) y seleccionando el DB subnet group creado para las subredes privadas.

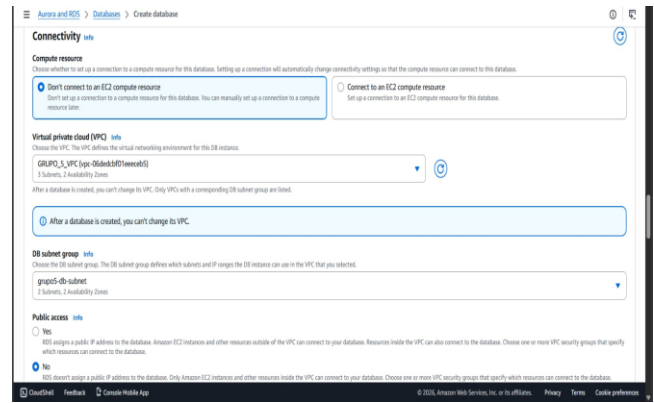


Figura 48. Configuración de conectividad de RDS: VPC del proyecto y acceso público deshabilitado.



Figura 49. Etiquetas (tags) opcionales asignadas al recurso RDS.

Finalmente, AWS confirmó la creación exitosa de la base grupo-5-rds, mostrándola en estado de configuración inicial.

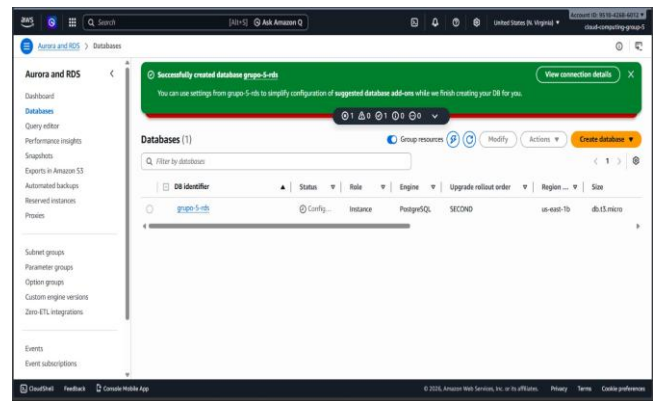


Figura 50. Mensaje de éxito: base de datos grupo-5-rds creada correctamente.

G. DB Subnet Group y reglas finales de RDS

Para que RDS pudiera utilizar las subredes privadas, se creó previamente un DB Subnet Group con el nombre grupo5-db-subnet, asociado a la VPC GRUPO_5_VPC.

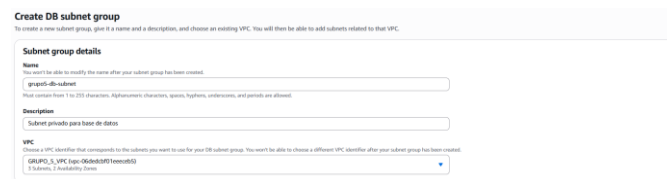


Figura 51. Formulario de creación del DB Subnet Group grupo5-db-subnet.

Se agregaron las dos subredes privadas (A en us-east-1a y B en us-east-1b) para cumplir el requisito multi-AZ de RDS.

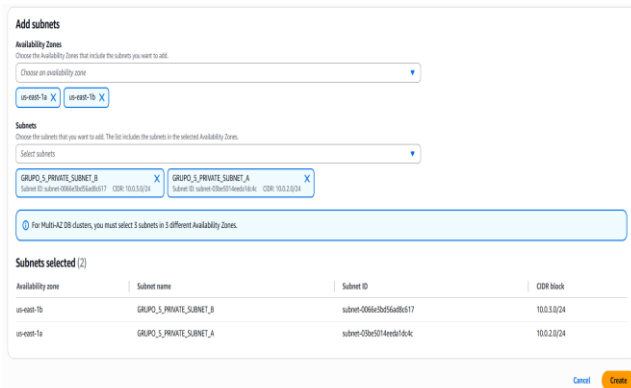


Figura 52. Asignación de las subredes privadas A y B al DB Subnet Group.

Posteriormente se ajustaron las reglas inbound del SG de RDS, autorizando exclusivamente al SG GRUPO_5_EC2_SG por el puerto 5432.

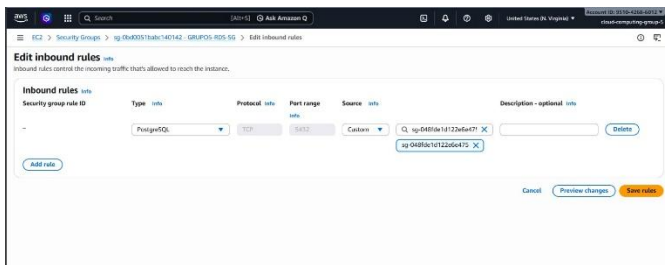


Figura 53. Edición de reglas inbound del SG GRUPO5-RDS-SG: PostgreSQL solo desde el SG de EC2.

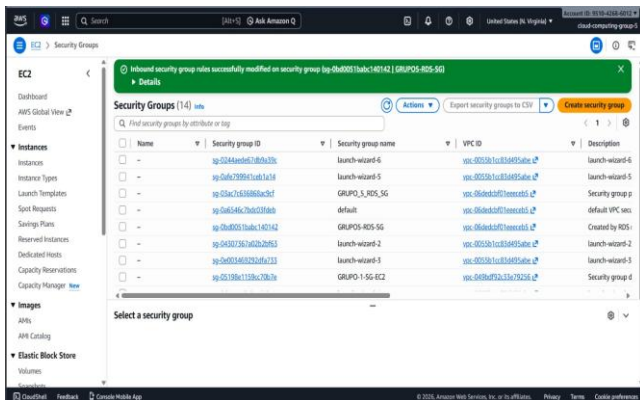


Figura 54. Confirmación de actualización exitosa de las reglas inbound del Security Group de RDS.

H. Validación de comunicación EC2 ↔ RDS

Para comprobar la comunicación EC2-RDS, se accedió nuevamente a la instancia EC2 mediante SSH desde Windows.

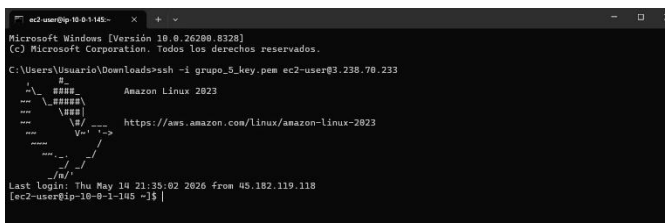


Figura 55. Conexión SSH a la instancia EC2 antes de instalar el cliente PostgreSQL.

Dentro de EC2 se instaló el cliente PostgreSQL 15 mediante dnf, dependencia necesaria para utilizar psql.

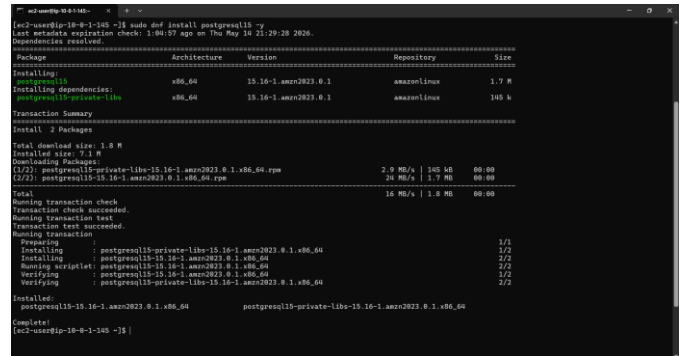


Figura 56. Instalación del cliente postgresql15 en la instancia EC2.

Se ejecutó psql contra el endpoint de Amazon RDS. La conexión fue exitosa, abriendo la consola interactiva de PostgreSQL bajo TLSv1.3.

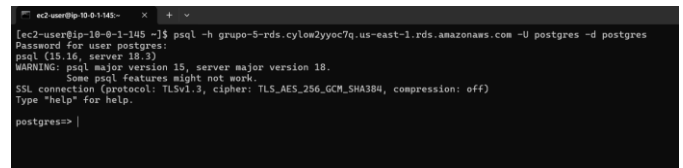


Figura 57. Conexión exitosa desde EC2 hacia RDS mediante psql sobre TLS.

Una vez dentro, se creó la tabla usuarios con los campos id (SERIAL PRIMARY KEY), nombre y correo (VARCHAR(100)).

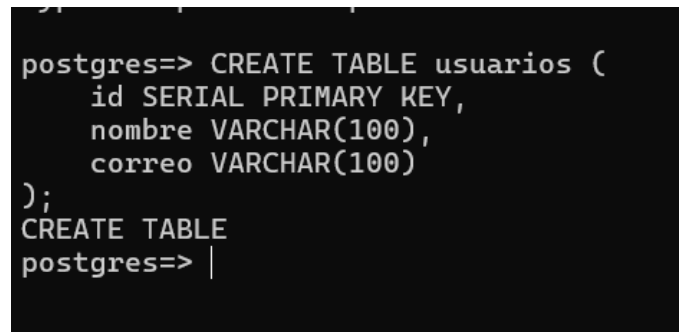


Figura 58. Creación de la tabla usuarios en la base de datos RDS.

Se verificó la creación mediante el comando \dt, confirmando la presencia de la tabla en el esquema público.

```
postgres=> \dt
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | usuarios | table | postgres
(1 row)

postgres=> |
```

Figura 59. Verificación de la tabla usuarios con el comando \dt.

I. Configuración del proyecto Java con PostgreSQL

Posteriormente se modificó el proyecto Java (descargado desde el repositorio base user-service-ci-cd) para conectarse a la base de datos. En el archivo src/main/resources/application.yml se definió la URL JDBC apuntando al endpoint de RDS, el usuario, la contraseña y el driver PostgreSQL.

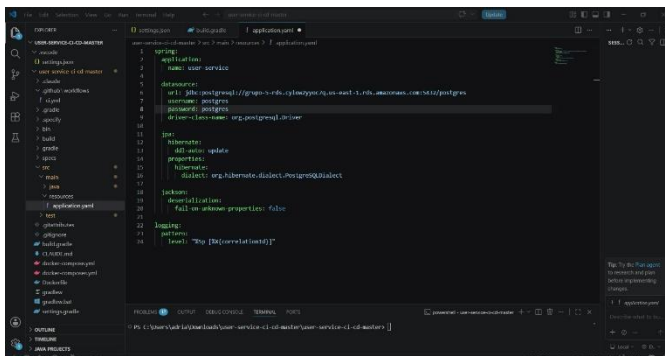


Figura 60. Archivo application.yml configurado con el datasource Amazon RDS y JPA.

También se añadió la dependencia del driver PostgreSQL (org.postgresql:postgresql:42.7.3) en el archivo build.gradle.

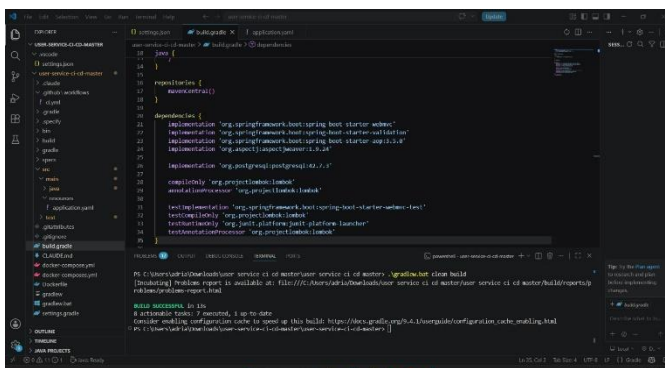


Figura 61. Dependencia del driver PostgreSQL declarada en build.gradle.

Antes de empaquetar, se ejecutó la aplicación Spring Boot localmente mediante el wrapper de Gradle (./gradlew bootRun), confirmando que Tomcat inicia en el puerto 8080.

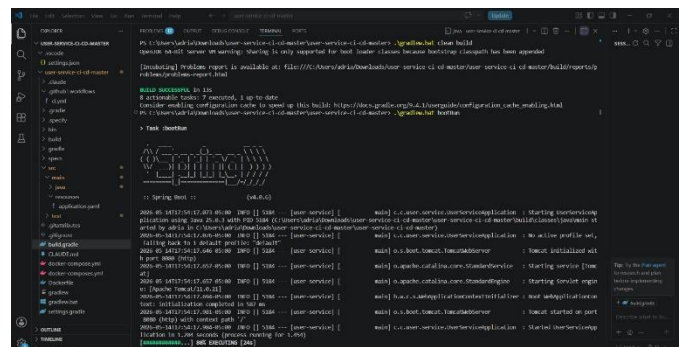


Figura 62. Spring Boot ejecutándose localmente con Tomcat en el puerto 8080.

J. Construcción y prueba local del proyecto

Desde PowerShell se realizó una petición POST al endpoint /api/v1/users mediante Invoke-WebRequest. La respuesta obtenida fue 201 Created con el cuerpo JSON del usuario registrado, demostrando el correcto funcionamiento del backend con la base de datos RDS.

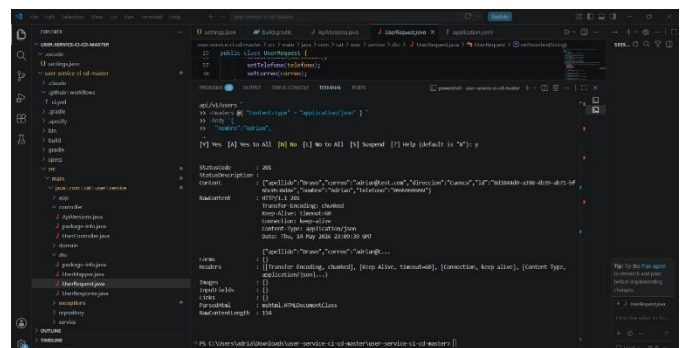


Figura 63. Prueba local del endpoint /api/v1/users con respuesta 201 Created.

K. Validación del bloqueo desde DBeaver

Uno de los requisitos críticos del proyecto era demostrar que la base de datos no podía ser accedida directamente desde una herramienta externa. Para ello se utilizó DBeaver. Primero se seleccionó el driver PostgreSQL.



Figura 64. Selección del driver PostgreSQL en DBeaver para intentar conectarse a RDS.

Se configuró la conexión con el endpoint público de RDS (grupo-5-rds.cylow2yyoc7q.us-east-1.rds.amazonaws.com), puerto 5432, usuario postgres y base postgres.

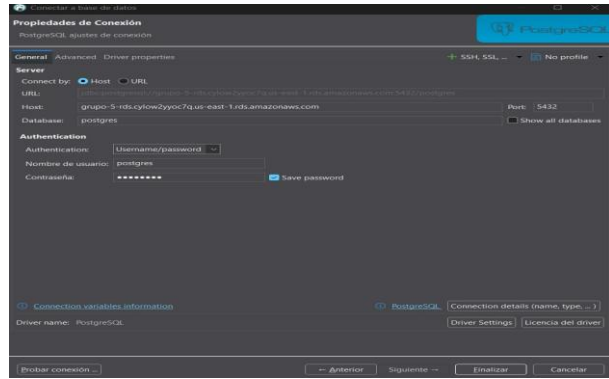


Figura 65. Configuración de conexión PostgreSQL en DBeaver hacia el endpoint de Amazon RDS.

Al intentar conectarse, DBeaver respondió con el error "Unknown host". Esto demostró que la base de datos no resuelve ni acepta tráfico desde fuera de la VPC, cumpliendo el requisito de aislamiento.

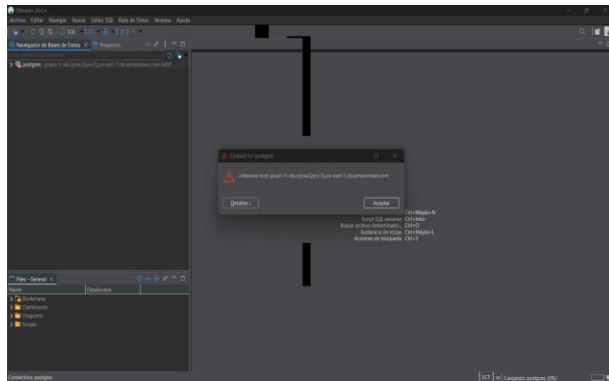


Figura 66. Conexión fallida desde DBeaver: el host no es accesible públicamente.

L. Pruebas locales con Docker Compose

A continuación se diseñó un archivo docker-compose.yml para levantar el sistema completo de forma containerizada. El archivo definió dos servicios (postgres y user-service), una red interna, healthchecks y volúmenes de datos.

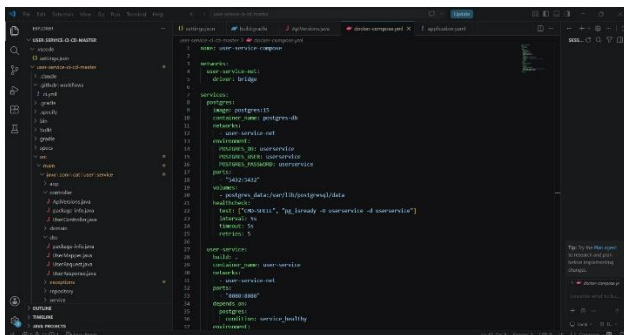


Figura 67. Archivo docker-compose.yml con los servicios postgres y user-service.

Al ejecutar docker compose up --build, Spring Boot inicializó correctamente, se conectó al contenedor PostgreSQL, ejecutó los checkpoints de la base y dejó disponible el servicio en el puerto 8080.

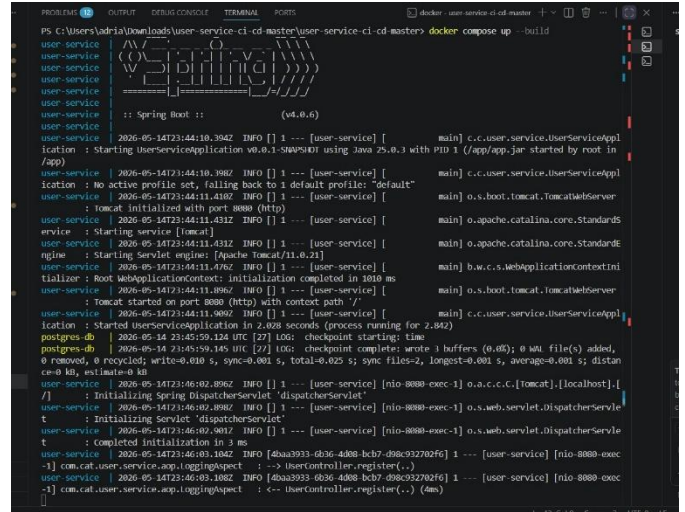


Figura 68. Logs de Spring Boot dentro del contenedor durante docker compose up.

Docker Desktop confirmó la ejecución del contenedor user-service con métricas de CPU y memoria activas.

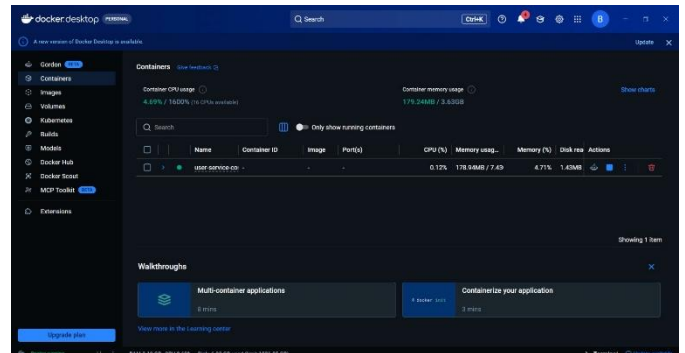


Figura 69. Contenedor user-service ejecutándose en Docker Desktop con métricas en tiempo real.

Una nueva prueba al endpoint local respondió con 201 Created, confirmando la operación del sistema containerizado.

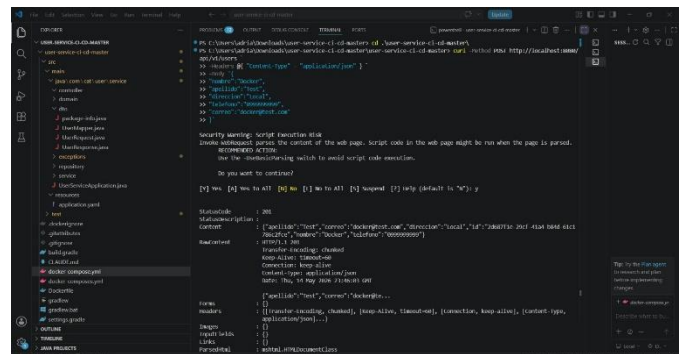


Figura 70. Prueba del endpoint dockerizado: respuesta 201 Created con datos del nuevo usuario.

M. Control de versiones y publicación en GitHub

Una vez validado el proyecto, se inicializó un repositorio Git local mediante `git init` y se verificó el estado de los archivos sin trackear.

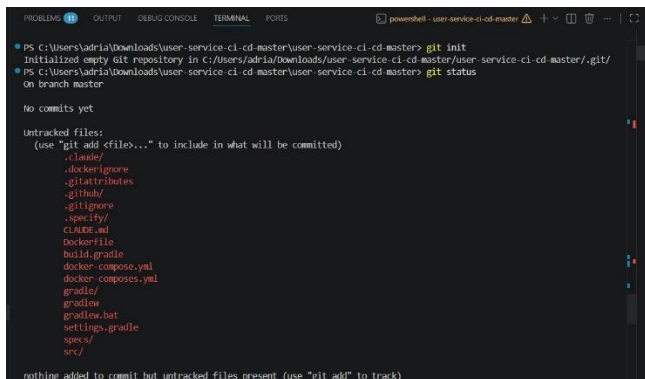


Figura 71. Inicialización del repositorio Git local y listado de archivos no trackeados.

Se agregaron todos los archivos al staging area con git add, aceptando las advertencias de finales de línea LF/CRLF en Windows.

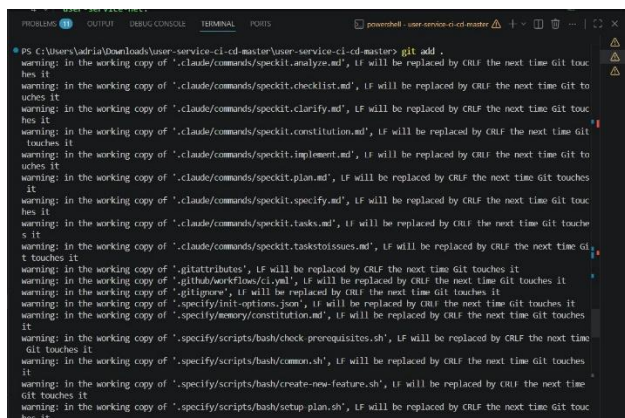


Figura 72. Ejecución de git add y advertencias de conversión de finales de línea CRLF.

Se realizó el commit inicial titulado "Proyecto Java con Docker Compose y PostgreSQL".

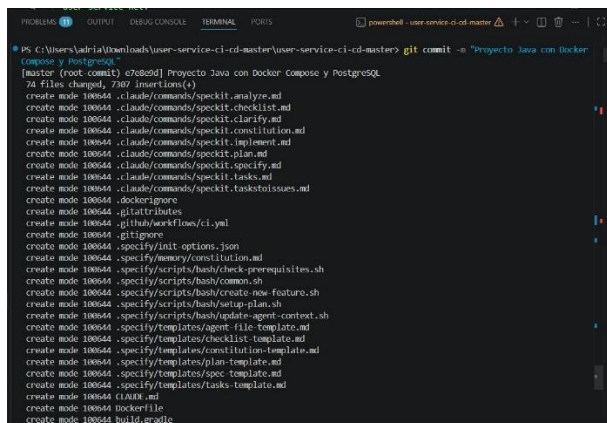


Figura 73. Commit inicial con 74 archivos y 7307 inserciones.

En GitHub se creó el repositorio público GRUPO_5_DEVOPS, siguiendo la convención de nomenclatura del enunciado.

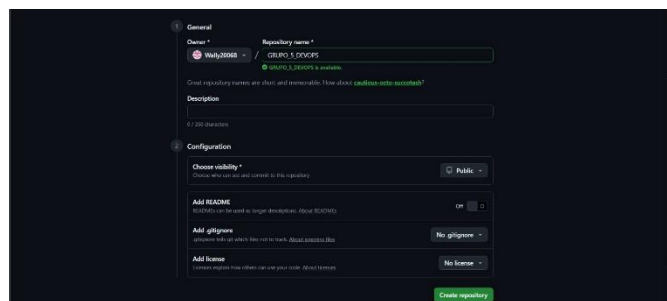


Figura 74. Creación del repositorio público GRUPO_5_DEVOPS en GitHub.

Se asignó el remoto origin y se ejecutó git push -u origin main, subiendo 119 objetos al repositorio remoto.

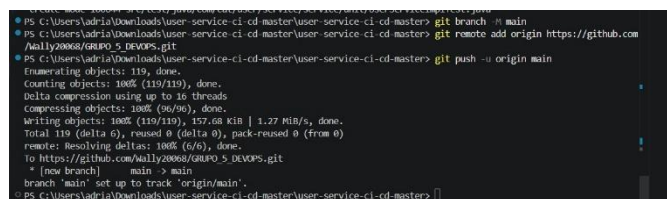


Figura 75. Push exitoso del repositorio local hacia GitHub.

El repositorio `GRUPO_5_DEVOPS` quedó accesible con todos los archivos del proyecto: `src/`, `build.gradle`, `Dockerfile`, `docker-compose.yml` y `.github/workflows`.

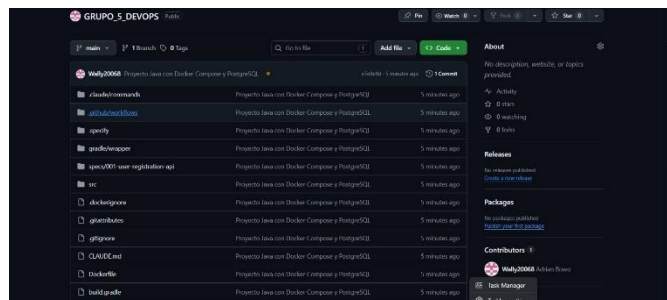


Figura 76. Repositorio GRUPO_5_DEVOPS visible en GitHub con todos los archivos del proyecto.

N. Configuración inicial de GitHub Actions

Dentro de la carpeta `.github/workflows` se creó el archivo `ci.yml`, definiendo el flujo "CI/CD Pipeline" activado por eventos push sobre la rama main. El primer job (build) configura Java 25 con la distribución Temurin y ejecuta `./gradlew build`.

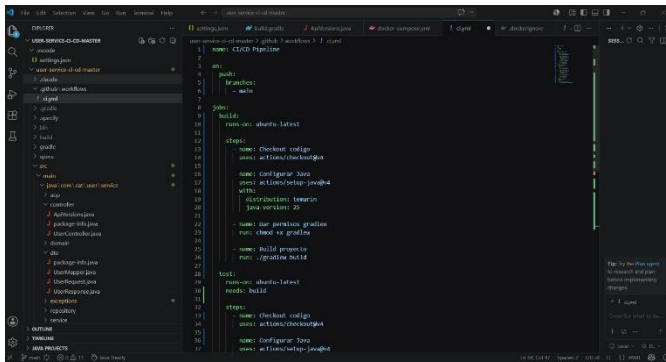


Figura 77. Definición inicial del workflow ci.yml con el job build.

En la primera versión, los jobs test y deploy se incluyeron también: test ejecuta ./gradlew test y deploy contenía un placeholder echo "Deploy hacia EC2 pendiente".

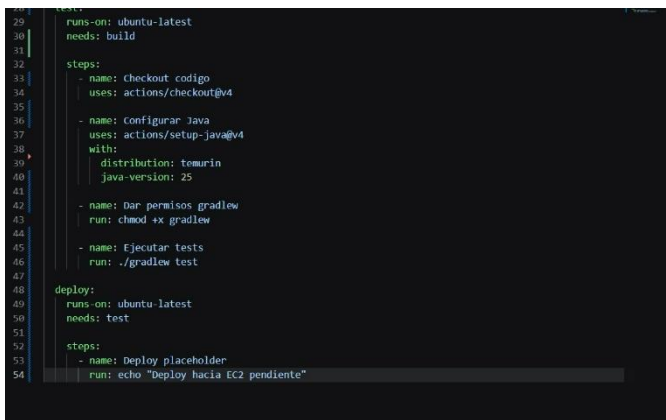


Figura 78. Jobs test y deploy en la primera versión del pipeline (deploy como placeholder).

Se verificó el estado del repositorio con git status, observando la modificación pendiente del archivo ci.yml.

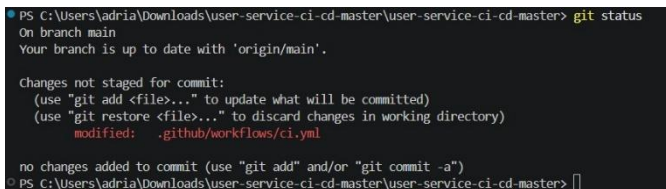


Figura 79. Estado del repositorio con la modificación del archivo ci.yml.

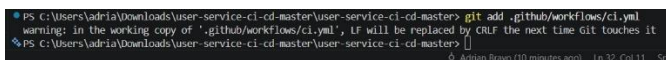


Figura 80. git add aplicado al archivo .github/workflows/ci.yml.

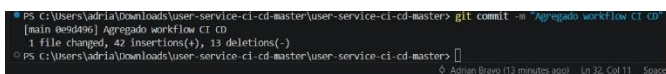


Figura 81. Commit "Agregado workflow CI CD" registrado en el repositorio local.

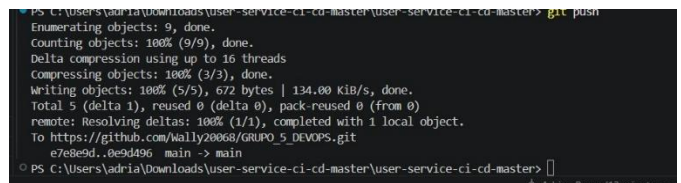


Figura 82. Push exitoso del commit con el workflow CI/CD hacia GitHub.

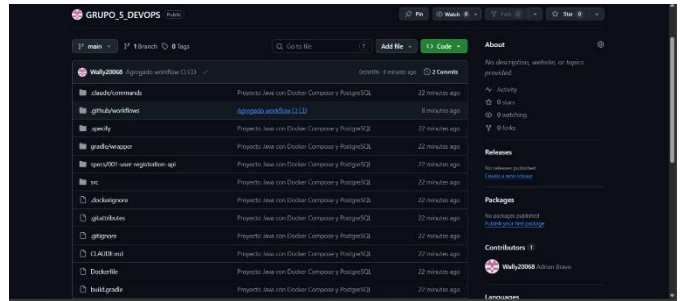


Figura 83. GitHub muestra el commit "Agregado workflow CI CD" en la rama main.

GitHub Actions ejecutó el pipeline y mostró ambos workflow runs (versión inicial y CI/CD agregado) con resultado Success.

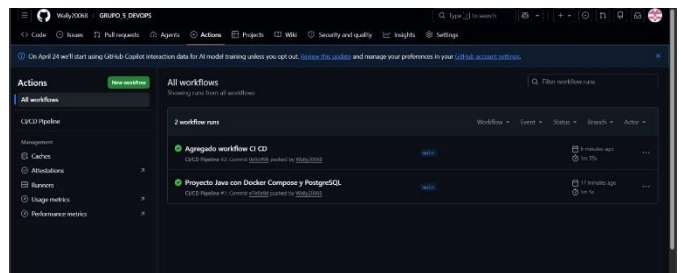


Figura 84. Lista de ejecuciones del pipeline en GitHub Actions con estado Success.

O. Configuración de Secrets en GitHub

Antes de implementar el deploy automático hacia EC2, se configuraron tres secrets en el repositorio. EC2_HOST contiene la dirección IPv4 pública de la instancia.

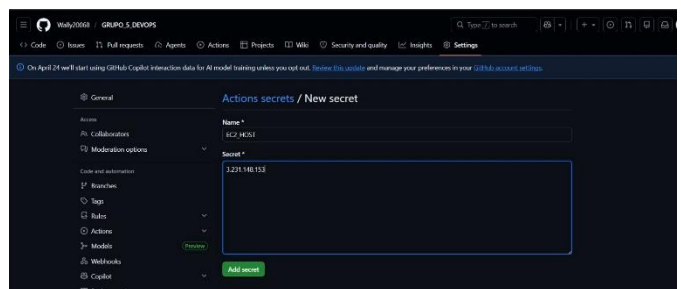


Figura 85. Secret EC2_HOST con la IP pública de la instancia EC2 (3.231.148.153).

EC2_KEY almacena la llave privada SSH en formato PEM, que el workflow utilizará para autenticarse en la instancia.

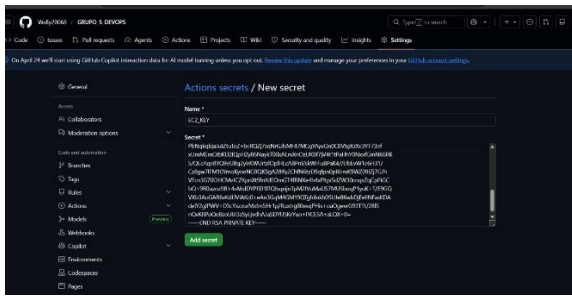


Figura 86. Secret EC2_KEY con la llave privada SSH del par grupo_5_key.

El listado final mostró los tres secrets configurados: EC2_HOST, EC2_KEY y EC2_USER.

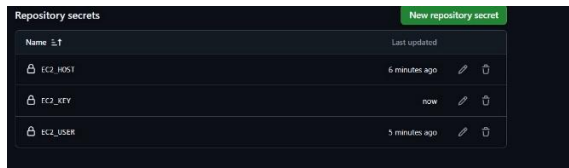


Figura 87. Listado de secrets del repositorio: EC2_HOST, EC2_KEY y EC2_USER.

P. Deploy automático hacia EC2

Se modificó nuevamente el archivo ci.yml para reemplazar el placeholder por un job de deploy real basado en la acción appleboy/ssh-action@v1.0.3.

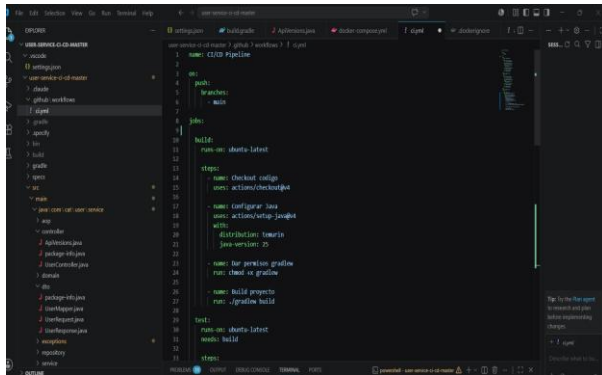


Figura 88. Workflow ci.yml actualizado con los jobs build y test definitivos.

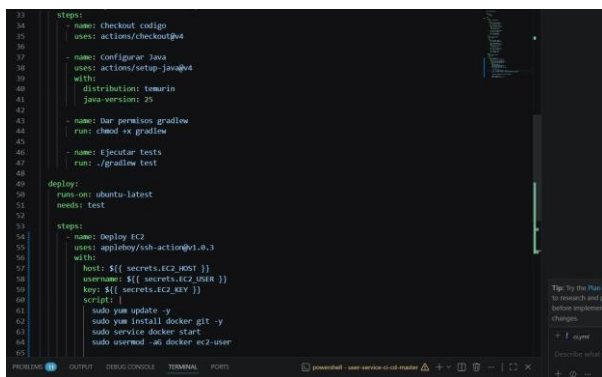


Figura 89. Job deploy con appleboy/ssh-action: parámetros host, username, key y script.

El script remoto actualiza el sistema, instala Docker y Git, inicia el servicio, clona el repositorio si no existe, entra a la carpeta del proyecto y ejecuta docker compose down seguido de docker compose up --build -d.

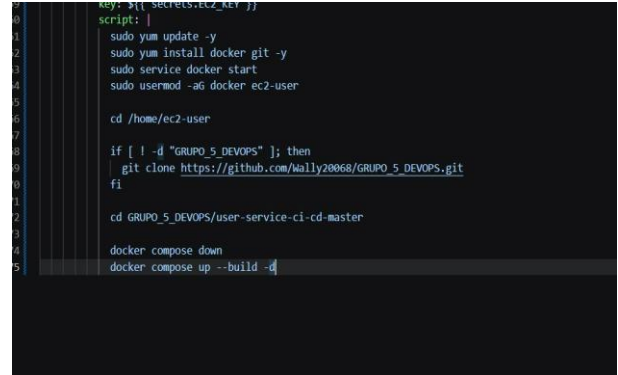


Figura 90. Detalle del script ejecutado vía SSH para desplegar la aplicación en EC2.

Se commiteó y pusheó la versión final del workflow.

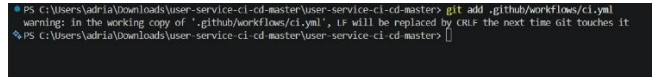


Figura 91. git add del archivo ci.yml con el deploy automático.

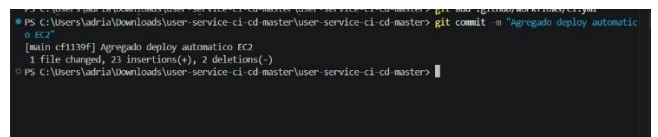


Figura 92. Commit "Agregado deploy automatico EC2" registrado en el repositorio.



Figura 93. Push del commit con el deploy automático hacia GitHub.

El nuevo workflow run "Agregado deploy automatico EC2" fue listado en GitHub Actions, junto con los anteriores, todos en estado Success.

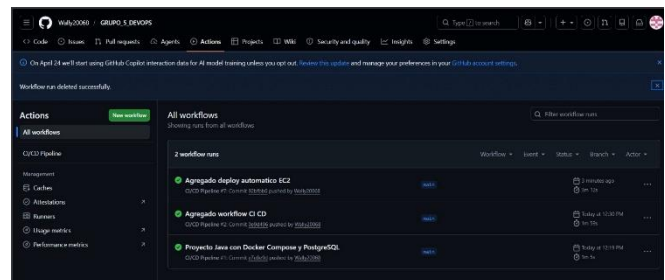


Figura 94. GitHub Actions con tres ejecuciones exitosas, incluyendo el deploy automático.

El detalle del pipeline mostró el grafo build → test → deploy completado en 3 m 12 s, con todos los jobs en verde.

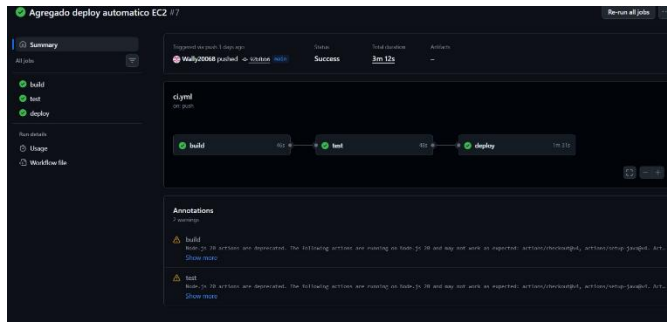


Figura 95. Detalle de la ejecución exitosa: build (46 s) → test (48 s) → deploy (1 m 31 s).

Al ingresar a la instancia EC2 vía SSH y ejecutar docker ps, se observó que los contenedores user-service (puerto 8080) y postgres-db (puerto 5432) estaban activos y en estado healthy.



Figura 96. docker ps en EC2: contenedores user-service y postgres-db ejecutándose correctamente.

Los logs del contenedor user-service confirmaron el arranque de Spring Boot v4.0.6 dentro de EC2.

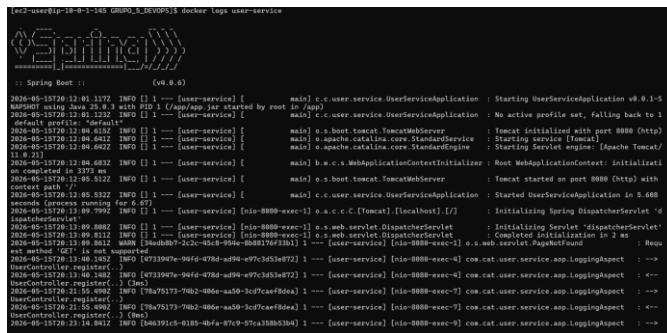


Figura 97. Logs de Spring Boot en el contenedor user-service desplegado en EC2.

Q. Validación final del endpoint público

Para cerrar el flujo DevOps, se realizaron pruebas desde Postman hacia el endpoint público `http://3.231.148.153:8080/api/v1/users`. La primera prueba registró al usuario Ana Pérez.

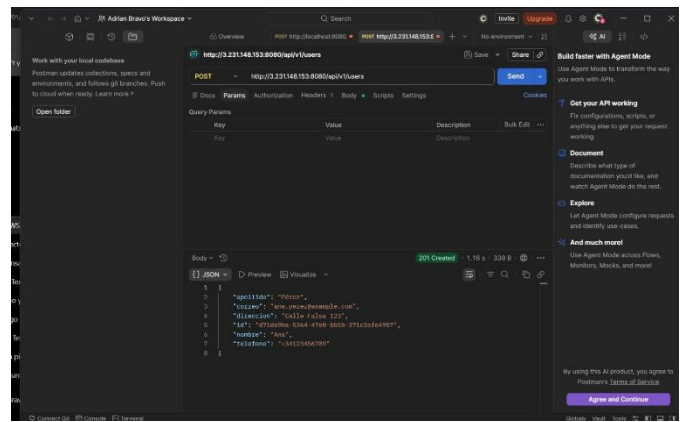


Figura 98. Postman: registro del usuario Ana Pérez en EC2 con respuesta 201 Created.

La segunda prueba registró a María González, con datos completos (nombre, apellido, teléfono, correo y dirección).

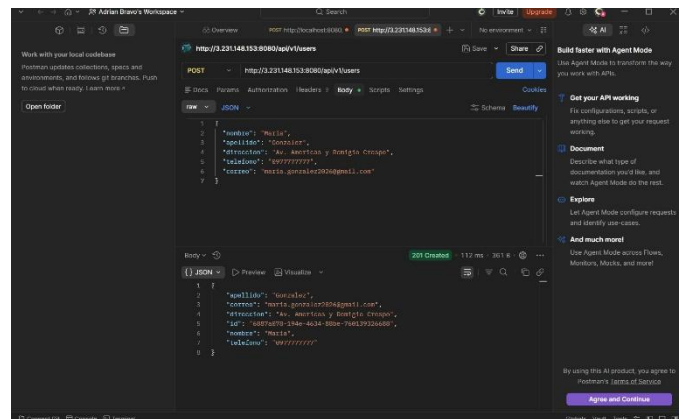


Figura 99. Postman: registro del usuario María González con respuesta 201 Created.

La tercera prueba registró a Carlos López, completando así el ciclo DevOps con tres usuarios registrados desde un cliente externo en la aplicación desplegada en AWS.

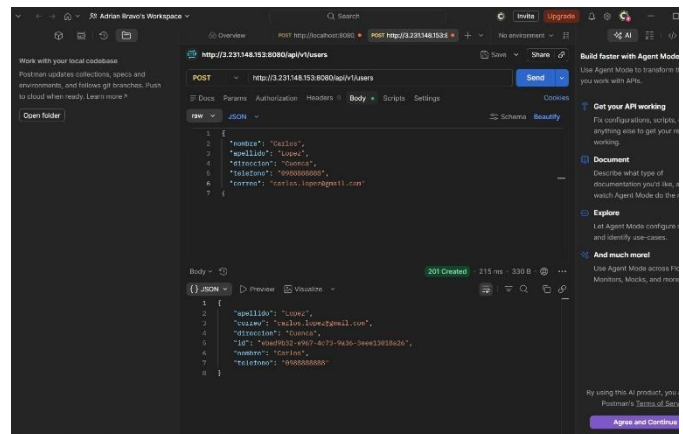


Figura 100. Postman: registro del usuario Carlos López con respuesta 201 Created.

V. RESULTADOS

Como resultado del proceso se obtuvo una arquitectura funcional en AWS. La VPC GRUPO_5_VPC permitió separar los recursos mediante una subred pública y dos subredes privadas. La instancia EC2 quedó ubicada en la subred pública, mientras que la base de datos RDS fue configurada dentro de las subredes privadas.

La comunicación entre EC2 y RDS fue validada mediante psql sobre TLS. Esta prueba confirmó que la configuración de red, rutas y Security Groups funcionaba correctamente. Adicionalmente, el intento de conexión desde DBeaver no fue exitoso, evidenciando que la base de datos no estaba expuesta públicamente.

El proyecto Java fue configurado para conectarse con PostgreSQL. La construcción con Gradle fue exitosa y las pruebas del endpoint local respondieron con 201 Created. También se comprobó el funcionamiento del sistema en Docker Compose, validando la comunicación entre el contenedor de la aplicación y el contenedor de PostgreSQL.

El repositorio GRUPO_5_DEVOPS fue creado y utilizado como fuente del pipeline. GitHub Actions ejecutó correctamente los jobs de construcción, pruebas y despliegue. Finalmente, el endpoint público de EC2 fue probado con Postman, obteniendo respuestas exitosas para los usuarios Ana, María y Carlos.

VI. DISCUSIÓN

La implementación demuestra la importancia de diseñar correctamente la infraestructura antes de desplegar una aplicación. Si la VPC, las subredes, las rutas o los grupos de seguridad se configuran incorrectamente, la aplicación puede quedar inaccesible o la base de datos puede quedar expuesta. En este caso, la separación entre recursos públicos y privados permitió mantener un equilibrio entre disponibilidad y seguridad.

El Security Group de RDS fue uno de los elementos más importantes. Al permitir tráfico PostgreSQL únicamente desde el Security Group de EC2, se evitó que cualquier computadora externa pudiera conectarse a la base de datos. Esta configuración fue clave para cumplir el requisito de bloquear el acceso desde DBeaver.

Docker Compose permitió probar la aplicación en un entorno local controlado antes del despliegue. Esta etapa ayudó a validar que el backend Java podía ejecutarse correctamente junto con PostgreSQL, reduciendo errores antes de llevar el sistema a AWS.

GitHub Actions permitió automatizar el proceso DevOps. Gracias al pipeline, cada cambio enviado a la rama principal pudo activar construcción, pruebas y despliegue. Esto convirtió el proceso en una secuencia repetible, verificable y menos dependiente de comandos manuales.

VII. CONCLUSIONES

- Se implementó exitosamente un proceso DevOps completo para un proyecto Java existente, integrando AWS, Amazon RDS, Docker Compose, GitHub y GitHub Actions.
- La arquitectura en AWS fue diseñada correctamente mediante una VPC personalizada, una subred pública para EC2 y subredes privadas para RDS, separando exposición y persistencia.
- La comunicación entre EC2 y RDS fue validada mediante el cliente PostgreSQL sobre TLS. La conexión exitosa confirmó que la red interna y las reglas de seguridad estaban correctamente configuradas.
- El acceso desde DBeaver no fue exitoso, cumpliendo con el requisito explícito de impedir conexiones externas directas hacia la base de datos.
- El proyecto Java fue modificado para conectarse a PostgreSQL y registrar usuarios. Las pruebas locales, las pruebas con Docker Compose y las pruebas finales en EC2 obtuvieron respuestas 201 Created.
- El pipeline de GitHub Actions permitió automatizar los jobs de build, test y deploy. Con ello, se logró un flujo CI/CD funcional para desplegar la aplicación en EC2 sin intervención manual.

VIII. RECOMENDACIONES

Se recomienda no mostrar contraseñas, claves privadas ni valores sensibles en capturas de pantalla. Si una llave privada fue expuesta durante la documentación, debe ser eliminada y reemplazada por una nueva.

Es recomendable utilizar variables de entorno o servicios de secretos para manejar credenciales de base de datos, evitando escribirlas directamente en archivos como application.yaml.

Para ambientes productivos se recomienda implementar HTTPS mediante AWS ACM y un Application Load Balancer, configurar monitoreo con CloudWatch, establecer políticas de respaldo automatizadas para RDS y separar ambientes de desarrollo, pruebas y producción.

IX. REFERENCIAS

- [1] Amazon Web Services. *What is Amazon VPC?* AWS Documentation. Disponible en: <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- [2] Amazon Web Services. *Enable internet access for a VPC using an internet gateway.* AWS Documentation. Disponible en: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html
- [3] Amazon Web Services. *Configure route tables.* AWS Documentation. Disponible en: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

- [4] Amazon Web Services. *Control traffic to your AWS resources using security groups*. AWS Documentation. Disponible en: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>
- [5] Amazon Web Services. *Amazon EC2 key pairs and Amazon EC2 instances*. AWS Documentation. Disponible en: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-key-pairs.html>
- [6] Amazon Web Services. *What is Amazon Relational Database Service?* AWS Documentation. Disponible en: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>
- [7] Docker Inc. *Get Docker*. Docker Documentation. Disponible en: <https://docs.docker.com/get-started/get-docker/>
- [8] GitHub. *GitHub Docs*. GitHub Documentation. Disponible en: <https://docs.github.com/>
- [9] GitHub. *GitHub Actions documentation*. GitHub Docs. Disponible en: <https://docs.github.com/en/actions>
- [10] Gradle Inc. *Gradle user manual*. Gradle Documentation. Disponible en: <https://docs.gradle.org/current/userguide/userguide.html>
- [11] Gradle Inc. *Building Java & JVM projects*. Gradle Documentation. Disponible en: https://docs.gradle.org/current/userguide/building_java_projects.html
- [12] Spring. *Spring Boot*. Spring Documentation. Disponible en: <https://spring.io/projects/spring-boot>